AquaSphere

Comprehensive System Documentation

A Web-Based Water Delivery E-Commerce System
with ML-Assisted Delivery Time Prediction

Technical record of the current implemented system

August 2026

Author

Tolentino, Lawrence Dave P.

Live application: https://aquasphere.up.railway.app/

GitHub repository: https://github.com/rorensdeyb/Aquasphere.git

Note: This is a second-year capstone project. Certain features remain incomplete or scaffolded. Known limitations are documented for transparency.

Table of Contents

1. Project Overview

AquaSphere is a web-based e-commerce system designed to streamline the ordering and delivery of purified drinking water products. The platform allows users to browse available water products, manage a shopping cart, place delivery orders, and track order status in real time. An integrated machine learning model predicts delivery time and shipping cost based on the customer's location, providing a data-driven logistics experience.

The project was developed in response to the growing demand for online water delivery services in local communities. Many water refill stations still rely on phone orders or walk-in transactions, which can lead to scheduling conflicts, manual delivery estimation, and limited order visibility for customers.

AquaSphere was developed as a second-year capstone project. While core functionality is implemented and deployed, certain features remain in an early or incomplete state. The system is presented here in its current working form, with known limitations documented for transparency.

Regional Coverage: The system currently covers the area around Region IV-A CALABARZON and does not cater to all places in the Philippines yet. This showcase demonstrates a potentially helpful system for water delivery businesses operating within this region.

Item	Current value
Project name	AquaSphere
Project type	Full-stack web application (e-commerce)
Intended users	Customers ordering water delivery; administrators managing products and orders
Primary live host	Railway (PHP 8.2 + PostgreSQL)
ML delivery model	Python scikit-learn (Random Forest / Linear Regression) via subprocess
Current status	Deployed and operational on free-tier services, with known limitations documented later
Project level	Second-year capstone project (ITST 304)

2. Statement of the Problem

Water delivery services in local communities face operational challenges related to manual order processing, inconsistent delivery time estimates, and limited customer visibility into order status. Customers often place orders through phone calls or social media messages, which can result in miscommunication, forgotten orders, and difficulty tracking delivery progress.

Without a centralized digital platform, water delivery businesses must rely on manual coordination between customers and delivery personnel. This process is time-consuming and prone to errors, particularly when estimating delivery times and calculating shipping fees based on distance.

Although e-commerce platforms exist for various product categories, few are specifically designed for the local water delivery niche, where delivery time prediction and location-based logistics are critical. This shows the need for a dedicated system that combines product browsing, order management, and machine learning–assisted delivery estimation in one integrated platform.

3. Objectives of the Project

3.1 General objective

To develop a web-based water delivery e-commerce system that allows customers to browse products, manage orders, and receive machine-learning–predicted delivery times and shipping fees based on their delivery location.

3.2 Specific objectives

- Design a responsive web application with user registration, email OTP verification, session login, and profile management.
- Develop a product catalog with image display, category filtering, and admin CRUD operations.
- Implement a shopping cart and checkout system with delivery address management and ML-based shipping fee calculation.
- Integrate Cash on Delivery as the active payment method and scaffold PayMongo digital payment for future activation.
- Develop an order management module with status tracking, history, and notification integration.
- Implement a machine learning model for delivery time prediction using geographic and order features.
- Develop an administrator dashboard for product, order, and user management.
- Deploy the system on Railway with PostgreSQL and maintain source code on GitHub.

4. Project Scope and Limitation

AquaSphere is a second-year capstone project developed within an academic timeframe. Core features including product browsing, cart management, checkout, order tracking, admin panel, and ML delivery prediction are implemented and functional.

The following features are not implemented or are incomplete in the current system:

- PayMongo digital payment integration is scaffolded but disabled ("Coming Soon").
- Product search is limited to category filtering; no full-text search.
- No product reviews, wishlists, promotional codes, or loyalty programs.
- ML model is trained on synthetic data; real-world accuracy not validated at scale.
- Notifications are limited to order-status updates only; no push or email notifications for non-auth flows.
- No mobile application; designed as a responsive web application.
- No automated test suite; testing is manual.

5. System Features and Functionalities

5.1 User Account Management

Registration with email OTP verification via Brevo, session-based login with HTTP-only cookies, password reset with OTP, email change with OTP verification, and profile management. Login lockout protection prevents brute-force attacks.

5.2 Product Catalog

Eight predefined water products seeded on first deployment. Products include image, label, description, price, category, and unit. Category filter bar on the dashboard. Admin CRUD for product management with image upload.

5.3 Shopping Cart and Checkout

Cart synchronized between localStorage and server database. Delivery address selection with province, city, barangay, and street fields. ML-based delivery fee calculation. Order summary with itemized costs.

5.4 Payment Processing

Cash on Delivery (COD) is the active payment method. PayMongo digital payment is scaffolded but currently disabled.

5.5 Order Management

Order status tracking (pending, confirmed, out for delivery, delivered, cancelled). Status history audit trail. Order details with items, delivery address, payment method, and delivery date range. Deep-link support for specific orders.

5.6 Notification System

Order-status notifications with badge count. Notification state persisted in database (notif_seen_at, notif_cleared_at) to survive logout/login. Bell click marks notifications as seen. Notification item click navigates to appropriate orders page.

5.7 ML Delivery Prediction

Random Forest or Linear Regression model predicting delivery time in minutes. Features: geographic coordinates, municipality, barangay, postal code, time of order, day of week, order size. PHP fallback when Python model unavailable. Delivery date range estimation (same-day, next-day, multi-day).

5.8 Administrator Dashboard

Sidebar-based admin interface with dashboard overview, user management, order management, and product management. Summary statistics (total users, orders, products, revenue). Charts for order and revenue trends.

5.9 Security Features

- Password hashing with PHP password_hash() (bcrypt).
- Session-based authentication with HTTP-only cookies.
- OTP verification for registration, password reset, and email changes.
- Input validation and sanitization on all API endpoints.
- Login lockout protection after repeated failed attempts.
- Role-based access control for admin functions.
- Dead session detection and automatic redirect to login.

6. Database Design

Production uses PostgreSQL via DATABASE_URL (Railway plugin). Local development uses SQLite when DATABASE_URL is not set. api/database.php handles connectivity using pg_* functions for PostgreSQL and SQLite3 class for local testing.

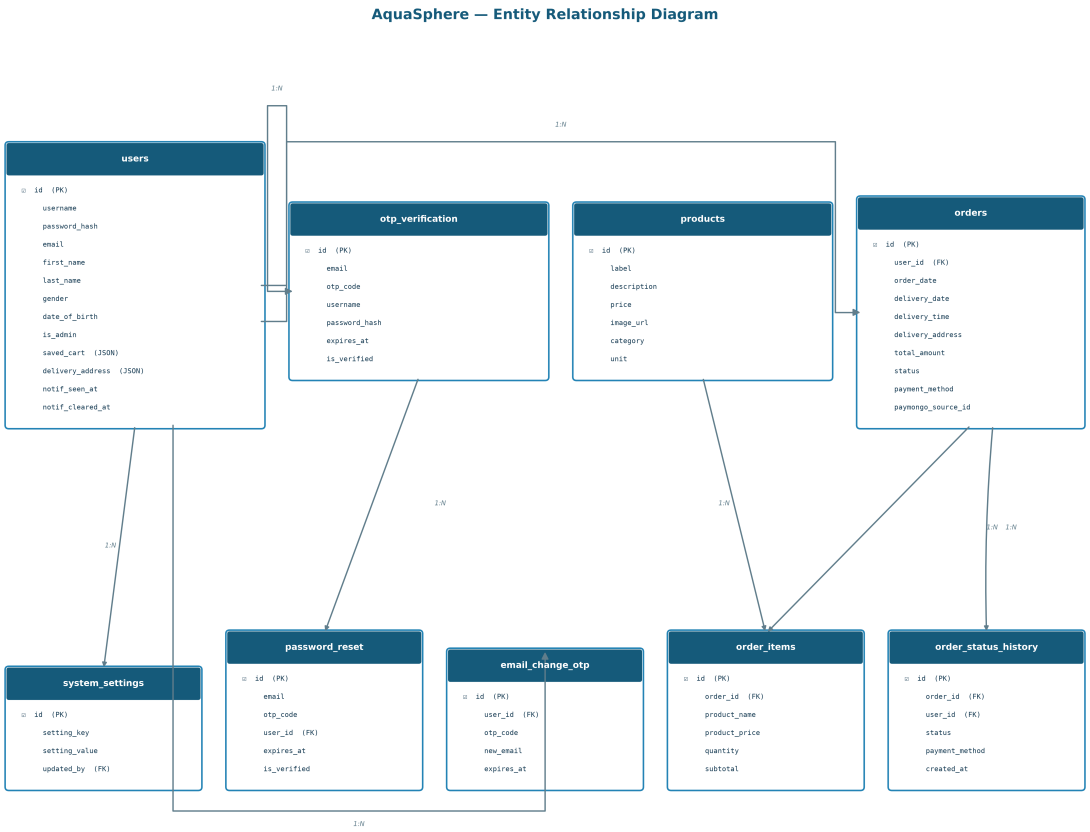


Figure 1. Entity Relationship Diagram showing core tables and their relationships.

Table	Role
users	Account credentials, profile fields, cart state, delivery address, notification timestamps, admin flag, suspension fields
otp_verification	Pending registration records with OTP codes and expiry timestamps
password_reset	Password reset OTP records linked to user_id
products	Water product catalog with label, description, price, image, category, unit
orders	Order records with user_id, delivery details, total amount, status, payment method
order_items	Individual line items within an order (product name, price, quantity, subtotal)
order_status_history	Audit trail of order status changes with timestamps
system_settings	Key-value configuration storage for admin-managed settings
email_change_otp	OTP verification for email change requests

A Railway volume is mounted at /data/uploads for persistent image storage. The uploads/ directory in the application root is symlinked to this volume on each request via database.php.

The database schema shown in database_schema.sql uses MySQL syntax for ERD documentation purposes. The actual production deployment uses PostgreSQL with equivalent table structures created by database.php.

7. Machine Learning Methodology

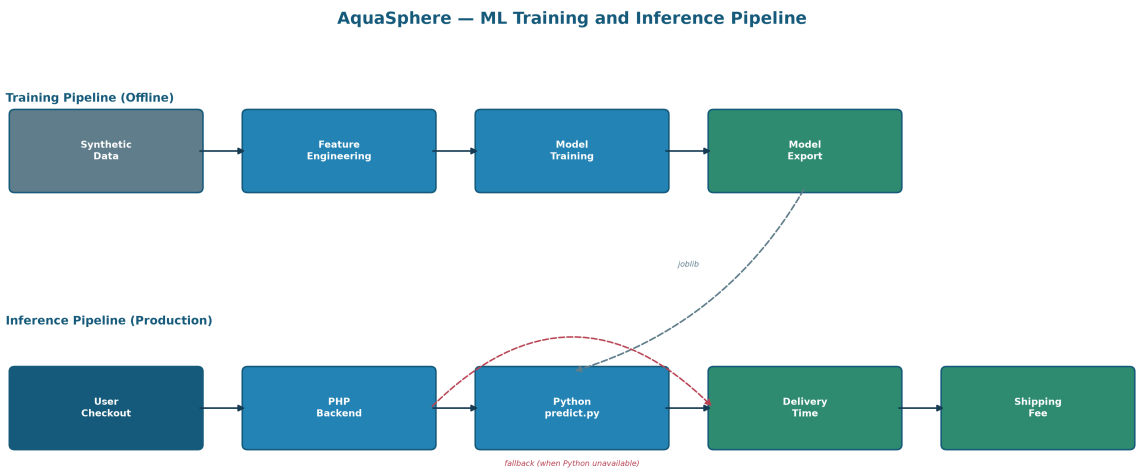


Figure 2. ML training and inference pipeline.

7.1 Problem definition

The ML module performs regression to predict delivery time in minutes given geographic and order features. The predicted time is used to calculate shipping fees and estimate delivery date ranges.

7.2 Features

Feature	Type	Description
distance_km	Numerical	Haversine distance from delivery hub (San Pablo City) to customer
latitude, longitude	Numerical	Customer delivery coordinates
municipality	Categorical	Municipality name (label-encoded)
barangay	Categorical	Barangay name (label-encoded)
postal_code	Categorical	Postal code (label-encoded)
time_of_order	Numerical	Hour of order (0–23)
day_of_week	Numerical	Day of week (0=Monday, 6=Sunday)
order_size	Numerical	Number of water bottles

7.3 Training process

Two models are trained and compared: Linear Regression and Random Forest Regressor. The model with higher R^2 on the test set is selected. Training uses synthetic data generated from San Pablo City geographic patterns. The pipeline includes label encoding of categorical features, 80/20 train/test split, and model export as joblib files.

7.4 Fallback behavior

When the Python model is unavailable, a PHP fallback calculates delivery time as: time = 15 + (distance × 2.5) + (order_size × 0.5), with a minimum of 20 minutes. Shipping fee = 50 + (time × 0.5). This ensures checkout remains functional without the ML model.

7.5 Deployment workflow

Training is performed offline: synthetic data generation (generate_synthetic_data.py), model training (train_model.py), model export (joblib). Production uses predict.py invoked via subprocess from PHP predict_delivery.php. The web application does not load the full training pipeline.

8. API Integration and External Services

8.1 Brevo Transactional Email

Used for registration OTP, password reset OTP, and email change OTP delivery. API key and sender configured in Brevo dashboard and stored as environment variables.

8.2 PayMongo (Scaffolded)

Payment gateway scaffolding exists with webhook handling and source creation endpoints. Currently disabled and displayed as "Coming Soon" on the payment page.

8.3 Chart.js

Used for order and revenue charts in the admin dashboard.

8.4 Bootstrap and Font Awesome

Bootstrap 5.3 provides responsive grid and components. Font Awesome 6.4 provides icons. Google Fonts (Inter) provides typography.

9. Deployment Process

9.1 Railway deployment

GitHub repository connected to Railway. PHP built-in server via Dockerfile: php -S 0.0.0.0:$PORT -t . PostgreSQL plugin provides DATABASE_URL. Volume mounted at /data/uploads for persistent image storage.

9.2 Docker configuration

Dockerfile based on php:8.2-cli with PostgreSQL extensions (pdo_pgsql, pgsql) and Python 3 for ML. Python virtual environment created during Docker build.

9.3 Environment variables

Variable	Purpose
DATABASE_URL	PostgreSQL connection string (injected by Railway)
BREVO_API_KEY	Brevo email API key
BREVO_SENDER_EMAIL	Brevo verified sender address
UPLOADS_DIR	Persistent image directory (/data/uploads)
PORT	HTTP listen port (provided by Railway)

10. Technologies Used

Category	Technology / Service
Frontend	HTML5, CSS3, JavaScript (ES6+), Bootstrap 5.3, Font Awesome 6.4, Chart.js, Google Fonts (Inter)
Backend	PHP 8.2 (built-in server locally, Docker in production)
Database	PostgreSQL on Railway (pg_* functions); SQLite locally (SQLite3 class)
Machine Learning	Python 3, scikit-learn (Random Forest, Linear Regression), pandas, numpy, joblib
Email / OTP	Brevo Transactional Email API

Category	Technology / Service
Payment	Cash on Delivery (active); PayMongo (scaffolded, disabled)
Security	PHP password_hash(), session cookies, OTP verification, input validation
Deployment	Railway (PHP + PostgreSQL + volume), Docker (php:8.2-cli)
Version Control	Git / GitHub

11. User Interface and Page Documentation

All pages follow the AquaSphere design system with the color palette: Deep Water (#155A7A), Aqua Blue (#2383B5), Water Accent (#5BC0EB), Light Aqua (#EAF7FC). Screenshots of live UI pages were not supplied with this documentation pass; placeholders mark where captures should be inserted.

11.1 Landing Page (index.html)

Purpose. Provide product overview, branding, and entry points to login/registration.

Access. Public. First page visitors see.

Main UI. AquaSphere branding, navbar with logo, Sign In button, Call to Action for products. Hero section with water theme.

Actions. Navigate to login, registration, or browse products.

Server interaction. No authentication required.

[Screenshot: Landing Page]

Placeholder pending a captured screenshot of Landing Page.

11.2 Registration (registration.html)

Purpose. Collect user details and create a pending registration with OTP verification.

Access. Public. New users.

Main UI. First name, last name, username, email, gender, birthday, password with strength requirements.

Actions. Submit registration form; receive OTP via email.

Server interaction. POST api/register; Brevo sends OTP.

[Screenshot: Registration Page]

Placeholder pending a captured screenshot of Registration Page.

11.3 OTP Verification (verify.html)

Purpose. Confirm the six-digit email OTP and activate the account.

Access. Public users who just registered.

Main UI. Six-digit input fields, resend option, target email display.

Actions. Verify OTP code; resend code; navigate to login.

Server interaction. POST api/verify_otp; POST api/resend_otp.

[Screenshot: OTP Verification Page]

Placeholder pending a captured screenshot of OTP Verification Page.

11.4 Login (login.html)

Purpose. Authenticate and start a session.

Access. Public. Registered users.

Main UI. Username/email and password fields, Sign In button, links to Register and Forgot Password.

Actions. Submit credentials; request password reset.

Server interaction. POST api/login; POST api/forgot_password; POST api/verify_reset_otp; POST api/reset_password.

[Screenshot: Login Page]

Placeholder pending a captured screenshot of Login Page.

11.5 Dashboard / Products (dashboard.html)

Purpose. Browse water products and add to cart.

Access. Authenticated users.

Main UI. Product grid with images, names, prices, Add to Cart buttons. Category filter bar. Navbar with cart/orders/notification badges.

Actions. Add items to cart; filter by category; navigate to cart, orders, profile.

Server interaction. GET api/get_products; GET api/get_current_user; GET api/user_state_get.

[Screenshot: Dashboard / Product Catalog]

Placeholder pending a captured screenshot of Dashboard / Product Catalog.

11.6 Shopping Cart (cart.html)

Purpose. Review and manage cart items before checkout.

Access. Authenticated users.

Main UI. Cart items list with quantity controls. Order summary with subtotal, delivery fee, total. Delivery address form with location dropdowns.

Actions. Adjust quantities; select items; fill delivery address; proceed to checkout.

Server interaction. GET/POST api/user_state_get/save; POST api/predict_delivery.

[Screenshot: Shopping Cart]

Placeholder pending a captured screenshot of Shopping Cart.

11.7 Payment (payment.html)

Purpose. Select payment method and confirm order.

Access. Authenticated users at checkout.

Main UI. Payment method cards: Cash on Delivery (active), PayMongo (Coming Soon). Order summary. Place Order button.

Actions. Select COD; confirm order placement.

Server interaction. POST api/create_order.

[Screenshot: Payment Page]

Placeholder pending a captured screenshot of Payment Page.

11.8 Order History (orders.html)

Purpose. View current and past orders.

Access. Authenticated users.

Main UI. Order cards with ID, date, status badge, total amount, payment method. Active orders displayed prominently.

Actions. View order details; click notification to see specific order.

Server interaction. GET api/get_orders; GET api/get_notifications.

[Screenshot: Order History]

Placeholder pending a captured screenshot of Order History.

11.9 Recent Orders (recent_orders.html)

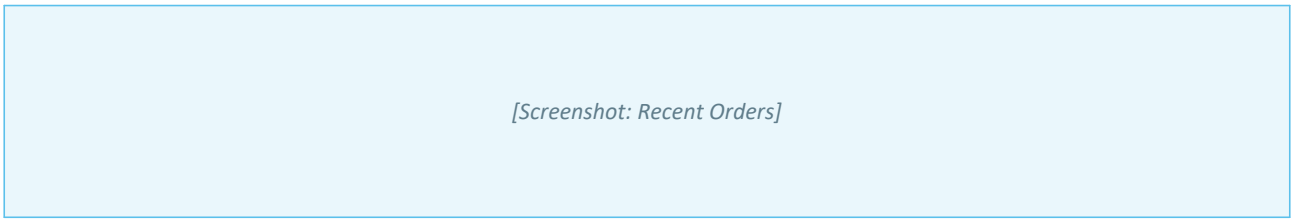
Purpose. View delivered, completed, and cancelled orders.

Access. Authenticated users.

Main UI. Order cards with status badges (green for delivered, red for cancelled). Cancelled On timestamp. Deep-link support.

Actions. View past order details.

Server interaction. GET api/get_orders (filtered by status).



Placeholder pending a captured screenshot of Recent Orders.

11.10 Profile (profile.html)

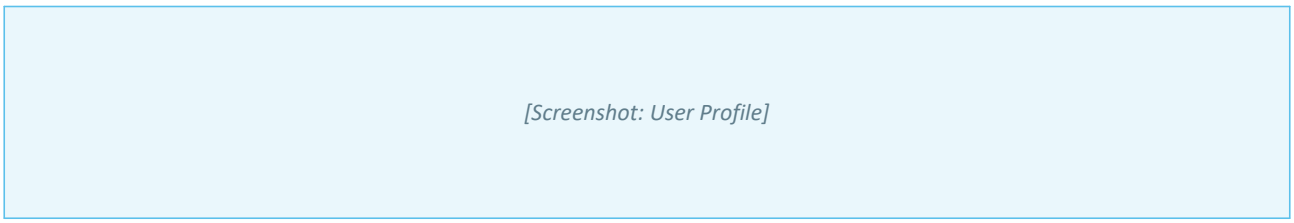
Purpose. View and manage account information.

Access. Authenticated users.

Main UI. User details (username, email, name, gender, DOB). Edit fields with green success styling. Password change with OTP. Email change with OTP modal.

Actions. Update profile; change password; change email; logout.

Server interaction. GET/POST api/get_current_user; POST api/update_profile; POST api/send_email_change_otp; POST api/verify_email_change_otp.



Placeholder pending a captured screenshot of User Profile.

11.11 Admin Dashboard (admin/dashboard.html)

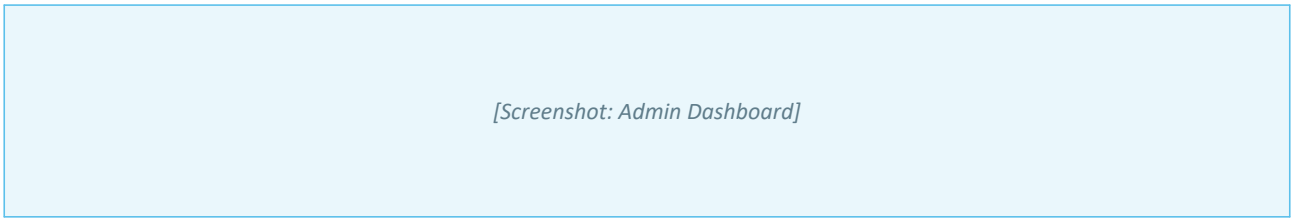
Purpose. Overview of system activity and business metrics.

Access. Authenticated admin user.

Main UI. Sidebar with AquaSphere gradient. Summary cards (users, orders, products, revenue). Charts for order trends.

Actions. Navigate to orders, users, products management.

Server interaction. GET api/get_orders; GET api/get_products; GET api/get_current_user.



Placeholder pending a captured screenshot of Admin Dashboard.

11.12 Admin Orders (admin/orders.html)

Purpose. View and manage all customer orders.

Access. Authenticated admin user.

Main UI. Order table with filtering by status. Order details modal. Status update controls.

Actions. Filter orders; view details; update status.

Server interaction. GET api/get_orders; POST api/update_order_status.

[Screenshot: Admin Orders]

Placeholder pending a captured screenshot of Admin Orders.

11.13 Admin Users (admin/users.html)

Purpose. View all registered user accounts.

Access. Authenticated admin user.

Main UI. User table with account details, registration date, order count. User detail modal.

Actions. View user details; monitor account activity.

Server interaction. GET api/get_users.

[Screenshot: Admin Users]

Placeholder pending a captured screenshot of Admin Users.

11.14 Admin Products (admin/products.html)

Purpose. Manage the water product catalog.

Access. Authenticated admin user.

Main UI. Product table with name, price, category, image. Add/Edit/Delete product modals with image upload.

Actions. Add new product; edit product; delete product.

Server interaction. GET/POST/PUT/DELETE api/admin/add_product, update_product, delete_product.

[Screenshot: Admin Products]

Placeholder pending a captured screenshot of Admin Products.

12. Conclusion and Recommendations

12.1 Conclusion

AquaSphere successfully demonstrates a web-based e-commerce system that applies PHP web development, PostgreSQL database integration, machine learning–based delivery time prediction, payment processing scaffolding, and online deployment. The system allows users to create secure accounts, browse and order water products, manage their shopping cart, place delivery orders, and track order status.

The project demonstrates practical application of machine learning in an e-commerce context through the delivery time prediction module, which uses geographic features to estimate delivery duration and shipping cost. The admin panel provides operational visibility for managing products, orders, and users.

12.2 Recommendations

- Activate the PayMongo digital payment integration to provide customers with digital payment options.
- Implement product search and filtering by name, product reviews, and promotional codes.
- Improve the ML model by training on real delivery data and deploying as a dedicated API service.
- Expand the notification system to include email notifications for order updates and push notifications for mobile.
- Develop automated tests for authentication, CRUD operations, and payment flows.
- Explore mobile application development for improved accessibility.

13. Project Links

Resource	URL
Live application	https://aquasphere.up.railway.app/
GitHub repository	https://github.com/rorensdeyb/Aquasphere.git

This document describes AquaSphere as an integrated system: a Railway-hosted PHP and PostgreSQL web application, a Python scikit-learn delivery time prediction model, Brevo email for OTP flows, and Cash on Delivery payment processing. Features that are scaffolded but incomplete (PayMongo) are listed as such so the record stays accurate for repository readers and future maintainers.